ATMC: working system, test results and current status

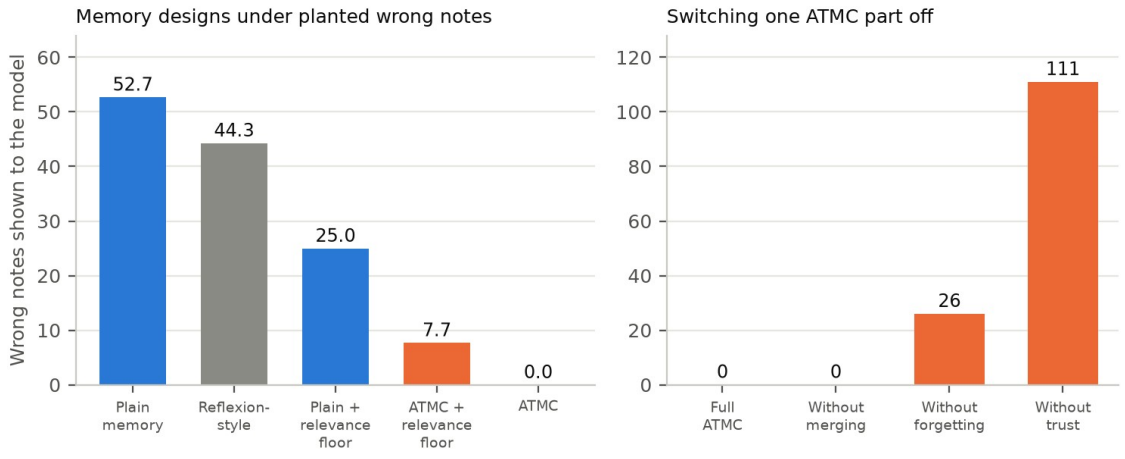
Self-evolving coding assistant. Summary for guide review.

1. What is working now: the Gemini-based assistant and its results

A web app where you ask Python questions. The assistant (Gemini 3.5 Flash-Lite) answers, the code runs in a sandbox against tests, and the result is saved in a **trust-scored memory (ATMC)** that it uses on later questions.

- **Working features:** chat assistant with sandboxed test runs; ATMC memory (trust, forgetting, merging); automatic answer checking when you give no tests; strategy evolution with automatic rollback; memory manager (search, pin, delete, export, import); Learning Lab (live no-memory vs memory demo); the same app also runs on a local model.
- **How it was tested:** the assistant is given 40 coding questions while wrong notes are **planted** in its memory, and compared with plain memory and a Reflexion-style memory (3 to 5 runs each).

Test (Gemini)	Result	What it means
Accuracy	No memory 89.2%, plain memory 89.2%, ATMC 88.3%	Gemini already solves about 89%, so memory cannot raise accuracy.
Wrong notes shown to the model	Plain memory **52.7**, Reflexion-style 44.3, ATMC **0.0** per run. Wrong notes actually used: 9.3 vs 0	**Trust scoring keeps wrong notes out of the model's view.**
Which ATMC part does it	Without trust: **111** wrong notes shown; without forgetting 26; without merging 0 (full ATMC 0). Same pattern on math (5 runs)	**Trust is the decisive part.**
Math problems	ATMC 100% vs plain memory 98.3%; wrong notes shown 0 vs 51	Same finding in a second domain.
Planner, reflection, weakness curriculum	90.0%, 90.8%, 83.3% vs 88.3% random order	No significant accuracy gain (ceiling effect).
Who gets the credit (wrong notes recorded as correct, 5 runs)	Crediting only cited notes 343 wrong notes shown; crediting all 351; **blaming every retrieved note after a failure: 106** (about 70% fewer); accuracy unchanged	Citation-based credit did not help; blame-all credit did.

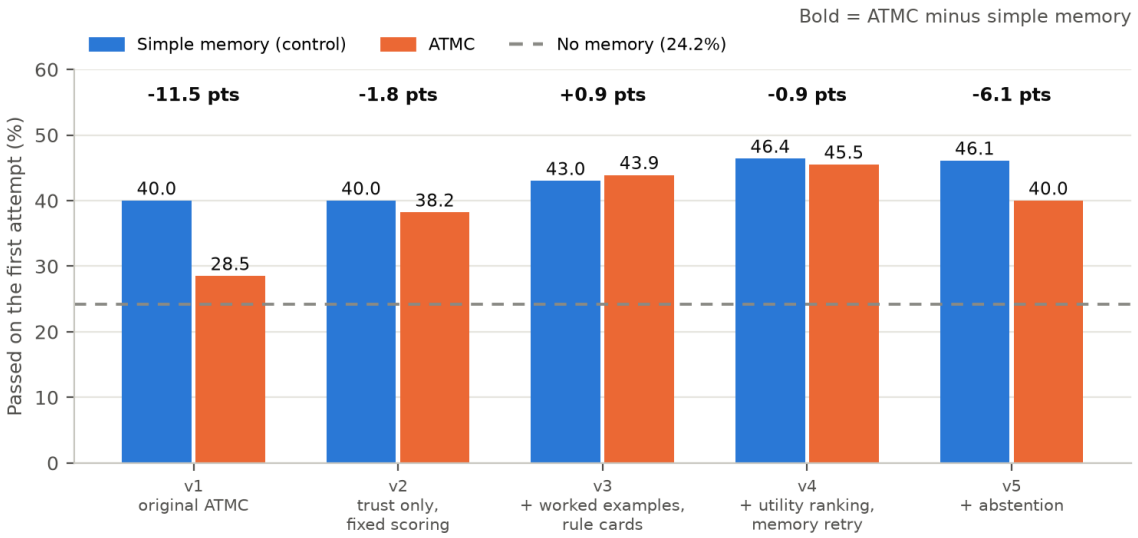


Wrong notes shown to the model per run (planted wrong notes). Left: memory designs. Right: ATMC with one part switched off.

In one line: on a strong model, ATMC's trust scoring does what it was designed for, but accuracy cannot improve because Gemini is already near its ceiling. That is why we moved to a smaller local model.

2. Versions v1 to v5 on the local model (Qwen 7B, 110 hidden-rule questions)

Each question hides a company rule (money in paise, DD-MM-YYYY dates, and so on). The model passes about **24%** with no memory and 100% of ordinary control questions. Numbers are the average of 3 runs on the same questions in the same order; the simple memory is always the control.



First-attempt pass rate. ATMC went from far behind simple memory (v1) to level with it (v3 and v4), but never ahead.

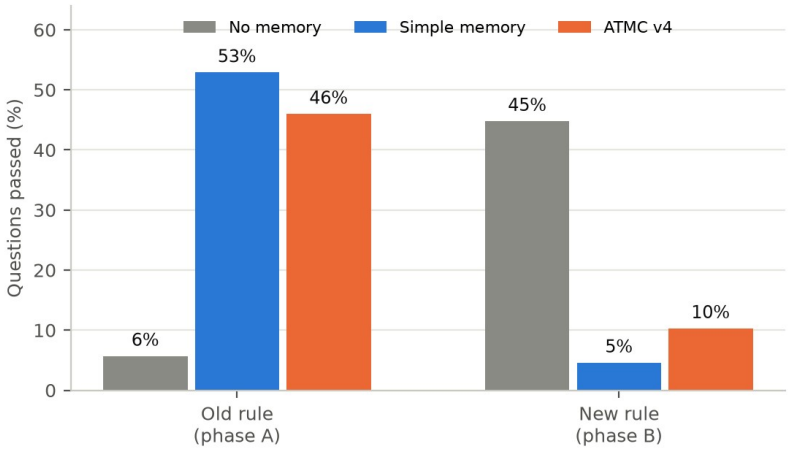
Ver.	What changed	Simple memory	ATMC	Difference	After one retry (simple / ATMC)	Verdict
v1	Original ATMC: trust, forgetting and merging	40.0%	28.5%	-11.5 (p < 0.001)	n/a	ATMC worse
v2	Diagnosis: trust only, failure notes scored fairly	40.0%	38.2%	-1.8 (p = 0.38)	n/a	Level. Forgetting alone cost 7.6 points
v3	Verified worked examples, one retry from the checker's evidence, rule cards	43.0%	43.9%	+0.9 (p = 0.79)	67.0% / 66.7%	Level
v4	Learned utility ranking, retrieval by failure type, retry that uses memory	46.4%	45.5%	-0.9 (p = 0.78)	68.8% / 68.5%	Level (+3 rule not met)
v5	Abstention: show only verified notes	46.1%	40.0%	-6.1 (p = 0.006)	66.4% / 65.2%	ATMC worse

Other tests run during these versions

Test	Result
Parts of ATMC added one at a time (v2)	Trust only 38.2%; plus forgetting **30.6%** (-7.6, p = 0.002); plus merging 30.9% (no change)
Checker wrongly passes 25% / 50% of wrong answers	Simple memory 34.8% / 31.8%; trust-only ATMC 35.5% / 34.2%. Differences +0.6 and +2.4, not significant: **trust did not protect**
Blame-all credit on the local model	Clean 29.4% (**-10.6** vs simple), 25% noise 30.9% (-3.9), 50% noise 31.8% (0.0)
Simple memory vs no memory (the core effect)	24.2% to 40.0% first attempt (**+15.8**, p < 0.0001); about **69%** after one retry

Rule-change test (run on v4)

29 questions are asked under an old rule and then again under the **opposite** rule, so every saved note about them is wrong. This is where trust and forgetting should matter.



Pass rate before and after the rule flips (average of 3 runs).

- **Simple memory collapses to 5%**, far below no memory (45%): stale notes mislead the model.
- **ATMC v4 reaches 10%** (+5.7 points over simple memory, p = 0.125, not significant). It also collapses, because trust updates are too slow for a rule change.

3. Current status of the project

Item	Status
Assistant app on Gemini and on the local Qwen model; Learning Lab (live: 25% vs 36% over 110 questions)	**Working**
Hidden-rule benchmark (110 questions), 200 automated tests	**Built, validated**
Experiments v1 to v5, safety tests, rule-change test on v4	**Complete**, results saved
v6: re-verification detector for rule changes (re-run stored solutions against the current tests; mark stale ones and their neighbours)	Code built and tested; **1 of 6 runs done, paused**, not yet run
Second, unseen rule-change stream (usernames, error handling) and 30 unseen held-out questions	Built; **reserved** for one final run each
Fine-tuning the model on its verified solutions	**Not started**
Paper, slides, README, novelty analysis	Paper has the Gemini results; **local-model results not yet added**
Version control	Nothing committed yet (about 70 changed or new files)

Where we stand: the assistant genuinely learns from its own failures (24% to 40% first attempt, about 69% after one retry) and trust scoring is shown to keep wrong notes out of memory on Gemini. What is **not** shown is that ATMC beats a simple memory on the local model: it ties at best across versions v3 to v4, and loses on v1 and v5. The rule-change test exposed a real weakness, and v6 is the planned fix.

Next: run v6 on the rule-change stream, then once on the unseen stream; one run on the 30 unseen questions; fine-tuning; then update the paper, slides and demo.

All numbers come from saved result files (pilot/results, docs/results); each experiment's prediction and verdict is in docs/experiments.